ENGR1050: Lecture 16

Simulating proportional control of a 1D helicopter

October 21, 2026



Agenda

- Briefly discuss last survey and final course corrections
- In our next lab (**meet in GM lab**) we will continue developing our helicopter model.
- To prepare, we will process our potentiometer data from the last lab and build a numerical ODE solver for the helicopter model.
- Using this data, we will design a controller policy to make the helicopter hover at a desired angle.

You will complete today's exercise in *Homework 4* - if you go quickly today you can get a head start.

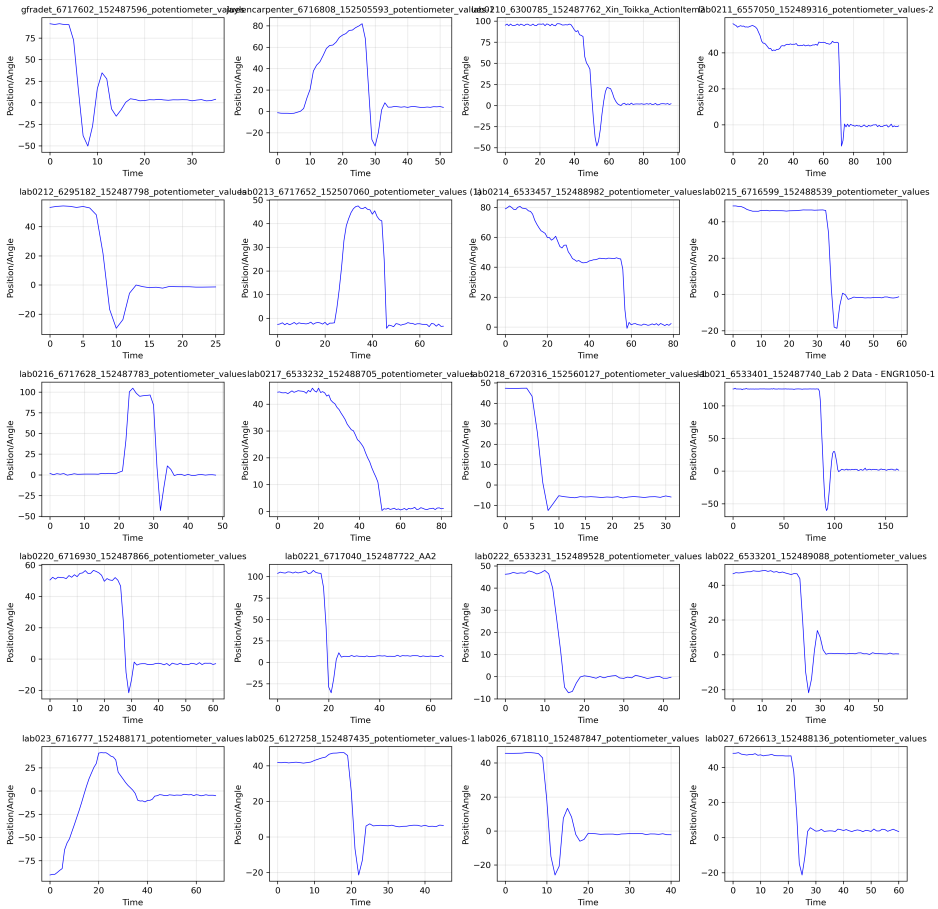
Survey responses.

Thanks all for the responses. A few things to note:

- Most (54.5%) find the course Challenging but manageable, with 15% finding it Too Challenging and the remainder finding it ok/easy.
  - I'm hearing too fast, so reducing material. I've refactored the simulation unit (no more partial differential equations) and instead we'll use and reuse our 'ODESolver' and spread out labs over multiple sessions.
- Happiness breakdown: Happy (49%), OK (40%), Unhappy (11%)
  - I'm happy with this, but brainstorming how to make this less bimodal.
  - Some trends: earlier survey notes 1/3 of the class haven't programmed before, and 10% "aren't comfortable with computers".
  - For those new to programming: it takes time; that's why the lowest exam in the class is dropped. Anyone who has been turning in in-class assignments and homework can get an A if they are proactive.
  - **Office hours!** I've been reaching out to folks trying to set up 1-1 tutoring sessions. Many folks new to programming are regulars in OH and are doing well - this is the most critical resource to succeed.
  - Beware using AI to "get through" assignments - this will give you trouble in exams.

Goal 1: Build an ODE model for the helicopter.





Recap: In the last lab we collected data from the helicopter model by running it freely and recording the angle over time using a potentiometer. Our goal is to build a model that we can use to design a controller to make the helicopter hover.

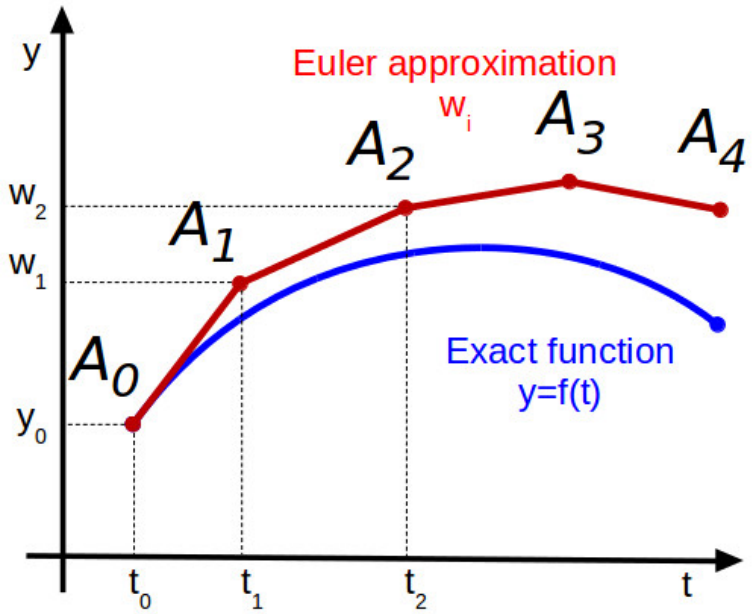
Numerical solution of ODEs

Many ODEs cannot be solved analytically, so we use numerical methods to approximate solutions. The simplest is the Explicit Euler method.

Definition

For the IVP $y'(t) = f(t, y)$ with $y(t_0) = y_0$ and step size $h > 0$, the Starting from t_0, y_0, Explicit Euler scheme updates via

$$t_{n+1} = t_n + h, \qquad y_{n+1} = y_n + h\, f(t_n, y_n),$$



Last lecture's ODE_solver class

```python
class ODE_solver:
    def __init__(self, t0, tf, y0, N):
        self.y0 = y0  # initial condition as numpy array
        self.t0 = t0  # initial time as float
        self.tf = tf  # final time as float
        self.N = N    # number of steps as integer
        self.h = (tf - t0) / float(N) # timestep size

        # We need to check what size y0 is and store that information
        self.dim = len(y0) #assumes that y0 is a list or array

        # Create a place to store the solution
        self.t_values = np.array([t0 + i*self.h for i in range(N+1)])  # time values
        self.solution = np.zeros((N+1, self.dim))  # solution array
        self.solution[0, :] = y0  # set initial condition

    def f(self,t,y):
        # Define the right-hand side of the ODE here, assuming a scalar t and a numpy array y
        # Example: dy/dt = -2y (for scalar y),
        # this function should return a numpy array of the same shape as y
        f = np.zeros_like(y)
        f[0] = -2 * y[0]  # Example for the first component
        return f

    def solve(self):
        # Implement explicit euler: y_n+1 = y_n + h*f(t_n, y_n)
        for n in range(self.N):
            t_n = self.t_values[n]
            y_n = self.solution[n, :]
            f_n = self.f(t_n, y_n)
            self.solution[n+1, :] = y_n + self.h * f_n
        return self.solution
```

Last lecture: nonlinear pendulum ODE

```
class nonlinearPendulum_solver(ODE_solver):
    def f(self,t,y):
        # Define the right-hand side of the ODE here, assuming a scalar t and a numpy array y
        # Example: dy/dt = -2y (for scalar y),
        # this function should return a numpy array of the same shape as y
        g = 9.81  # acceleration due to gravity
        L = 1.0   # length of the pendulum

        f = np.zeros_like(y)
        f[0] = y[1]
        f[1] = -g/L * np.sin(y[0])
        return f
```

- We can solve new ODEs by deriving child classes that override the `f` method.
- This class implements the nonlinear pendulum ODE as a system of first-order ODEs, we'll add friction and forcing to this today.
- In today's lecture we will change parameters - that amounts to modifying the `f` method. You might do this by making `f` take additional parameters as an input.

Last lecture: nonlinear pendulum

- The nonlinear pendulum is governed by the ODE

$$\theta''(t) + \frac{g}{L}\sin(\theta(t)) = 0,$$

  where $\theta(t)$ is the angle of the pendulum from vertical, g is the acceleration due to gravity, and L is the length of the pendulum.
- **Our code assumes first-order ODEs:** We must convert it to a system of first-order ODEs by defining $\omega = \theta'$:

$$\begin{cases} \theta' = \omega, \\ \omega' = -\frac{g}{L}\sin(\theta). \end{cases}$$

- We implemented this system in our `ODE_solver` class.

This time: adding friction and adding forcing from the fan

- To better model the helicopter, we add a friction term proportional to angular velocity:

$$\theta''(t) + b\theta'(t) + \frac{g}{L}\sin(\theta(t)) = 0,$$

  where b is a friction coefficient. This models air resistance and friction in the pivot, and damps out oscillations. The parameter b is something we will tune to match our data.
- We also add a forcing term from the fan:

$$\theta''(t) + b\theta'(t) + \frac{g}{L}\sin(\theta(t)) = \mathbf{T},$$

  where $\mathbf{T}$ is the torque applied by the fan. We'll discuss how to design $\mathbf{T}$ to make the helicopter hover at a target angle θ_{target}.
- To get this into our simulator, we again convert to first-order ODEs by defining $\omega = \theta'$:

$$\begin{cases} \theta' = \omega, \\ \omega' = -b\omega - \frac{g}{L}\sin(\theta) + \mathbf{T}. \end{cases}$$

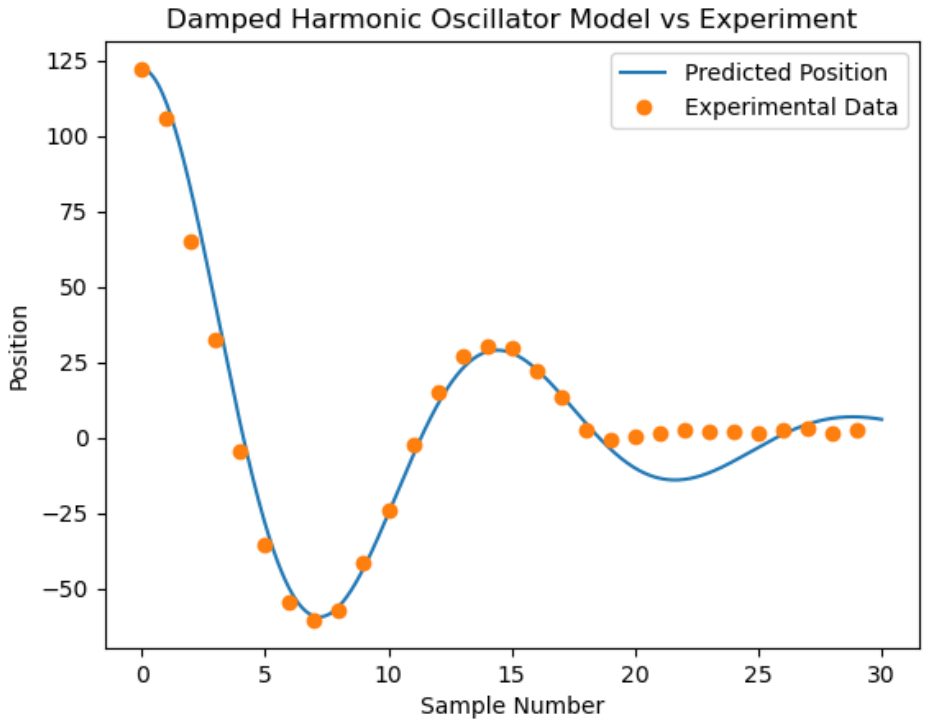
Modifying our ODE_solver to include friction and forcing

```
class nonlinearPendulum_solver(ODE_solver):
    def __init__(self, t0, tf, y0, N,b,K):
        super().__init__(t0, tf, y0, N)
        # Additional parameters for friction and forcing
        self.b = b  # friction coefficient
        self.K = K  # g/L term
    def f(self,t,y):
        # Define the right-hand side of the ODE here, assuming a scalar t and a numpy array y
        # b and K are specified as inputs to make it easy to play with them
        g = 9.81  # acceleration due to gravity
        L = 1.0   # length of the pendulum
        T = 0.0   # Forcing term to add torque (0 for now)
        f = np.zeros_like(y)
        f[0] = y[1]
        f[1] =  - self.b*y[1] - self.K*np.sin(y[0]) + T
        return f

#Example solve - solve with b = 0.1 and g/L = 9.81
pendulum = nonlinearPendulum_solver(t0=0, tf=10, y0=np.array([np.pi/2, 0]), N=10000,b=0.1,K=9.81)
solution = pendulum.solve()
```

- We can solve new ODEs by deriving child classes that override the `f` method.
- This class implements the nonlinear pendulum ODE as a system of first-order ODEs, we'll add friction and forcing to this today.
- In today's lecture we will change parameters - that amounts to modifying the `f` method. You might do this by making `f` take

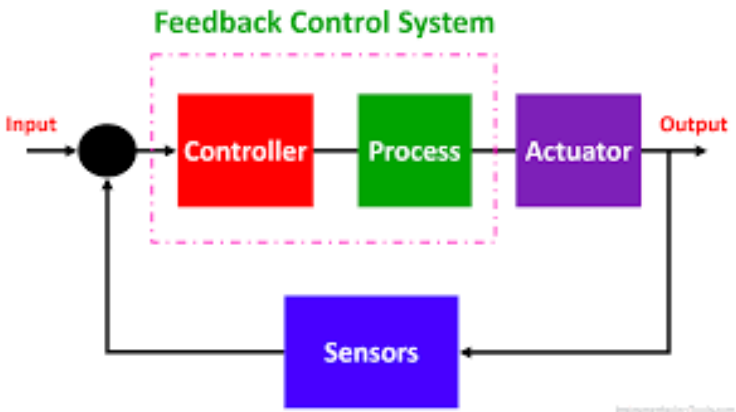
Task 1: Calibrate model to our experimental data without forcing



$$\begin{cases} \theta' = \omega, \\ \omega' = -b\omega - K\sin(\theta). \end{cases}$$

Hint: Need to match this model to data by tuning b and K. You could either get K from physical parameters $K = g/L$ or treat it as a fitting parameter. My advice is to fix $b = 0$ and tune K to match the frequency of the oscillation, and then tune b to match the decay of the oscillation amplitude.

Feedback Control Examples



- **Process:** Physical system (helicopter).
- **Actuator:** Applies power to fan.
- **Sensor:** Potentiometer $\rightarrow$ angle measurement
- **Controller:** Chooses power to fan based on angle measurement.

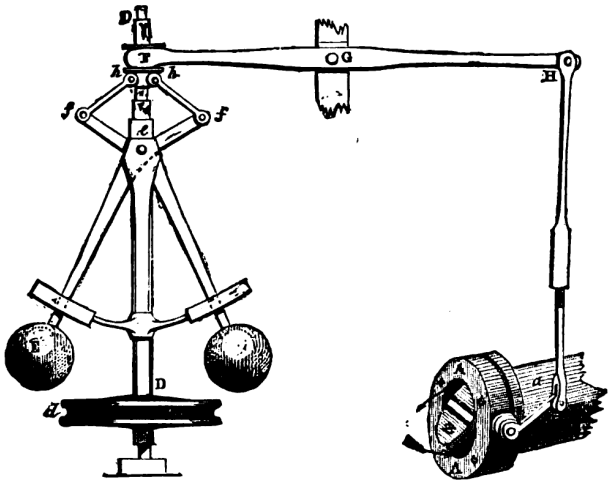
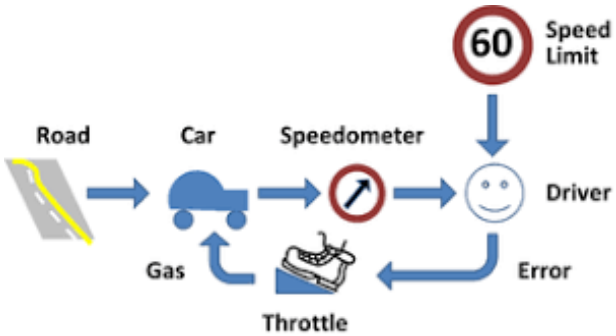
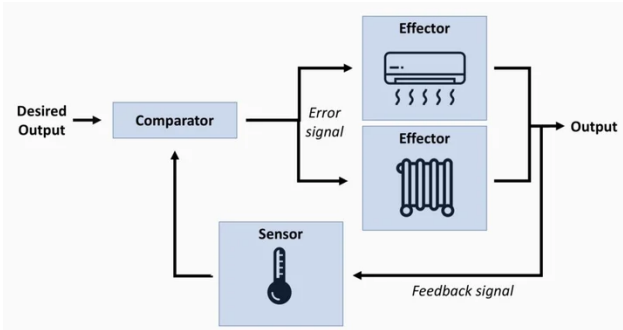


FIG. 4.—*Governor and Throttle-Valve.*





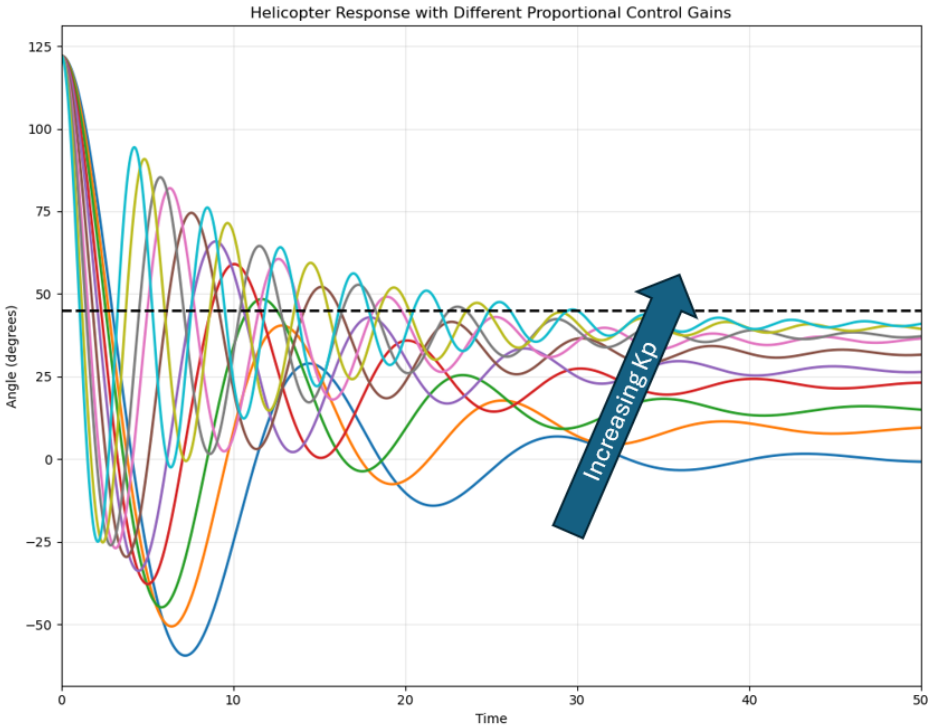
Proportional control to make the helicopter hover

- Now that we have a model that matches the data, we can use it to design a control policy to make the helicopter hover at a desired angle θ_{target}.
- We will use **proportional control**, where the torque applied by the fan is proportional to the error between the current angle and the target angle:

$$\mathbf{T} = CK_p(\theta_{target} - \theta),$$

  where K_p is the proportional gain and C is a constant that models the aerodynamics of the helicopter.
- By tuning K_p, we can control how aggressively the helicopter responds to deviations from the target angle.

Task 2: Calibrating controller to hit a target angle



$$\begin{cases} \theta' = \omega, \\ \omega' = -b\omega - K\sin(\theta) + \mathbf{T}, \\ \mathbf{T} = CK_p(\theta_{target} - \theta). \end{cases}$$

Fixing $C = 1$, you can play with different values of K_p to hit a target hover angle of 45°. How close can you get to the target angle in steady state?

Today

There were a few new things today, but at the end of the day you have two tasks:

- Take the pendulum model and run it for different choices of parameters until you get numbers that match the experimental data.
- Add the forcing term and look at how the trajectories change for different K_p.
- There will be time in the next lab to get extra help if you have trouble with this today.
- The next homework will gently extend this, showing how to derive a control that will hit the angle that we want smack on. If you follow what we did today, this will be a simple one line modification to your code from today.